FinTech Transaction and Fraud Risk Analysis

1. Project Overview

- Fintech Transaction and Fraud Risk Analysis is a data analytics project that analyze financial Transaction, Understanding the customer behavior and perform fraud risk analysis.
- The Project focuses on understanding transaction patterns, transaction amount, risk levels, and fraudulent transactions.
- It also analyzes factors such as transaction category, transaction status, credit score, KYC level, bank and device type.
- The data is cleaned and analysis using **Python in Google Colab** and the results are visualized using **Power BI**.
- The main goal is to **identify fraud and risk patterns and provide useful business insights for better financial risk management**.

2. Project Objective

- To analyze financial transaction and identify key transaction patterns and trends.
- To evaluate transaction amounts, types, categories and customer behavior.
- To identify high-risk transactions using risk-related factors.
- To analyze fraudulent transaction and identify patterns that may help in fraud detection.
- To generate actionable insights to support data-driven and improve financial risk and fraud management.
- Generate business insights and recommendations.

3. Data Source

Hugging face - FinTech
https://huggingface.co/datasets/hari7261/FINTECH-DETAILS-DATASET

Dataset: FinTech Transaction Dataset

Domain: Financial Transaction & Fraud Risk Analysis

Dataset Size: 5000 rows & 21 columns

Key Features: Transaction ID, Customer ID, Age, City, Country, Transaction Type, Category, Amount, Bank Name, Device Type, Transaction Status, Credit Score, KYC Level, Risk Level, Is Fraud, etc.

4. Problem Statement

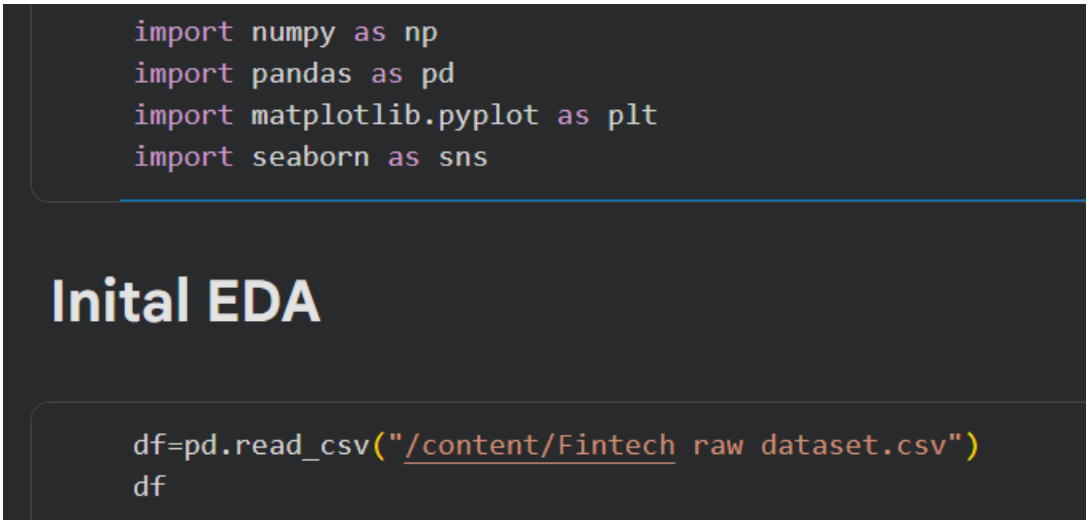
Financial transaction datasets contain large amounts of information, making it difficult to identify transaction patterns, high-risk activities, and fraudulent transactions manually. This project analyzes transaction and customer-related data to identify important fraud and risk patterns and provide data-driven insights for better financial risk management.

5. Data Cleaning & Pre-processing

Data cleaning and preprocessing were performed using **Python in Google Colab**. The raw FinTech transaction dataset was examined and prepared for further analysis.

The following steps were performed:

- Loaded the raw dataset using **Pandas**.



Then I performed initial EDA to understand the dataset

- Checked the first few records using head () to understand the dataset.

```
  print(df.head())                                ##To display top 5
rows
```

- Checked the last five records using tail () to understand the dataset

```
  print(df.tail())                                ##To display last 5 rows
```

- Checked the number of **rows and columns** using shape.

```
print(df.shape)                    ##To display number of rows and columns in the dataset

(5000, 21)
```

- Examined column names and **data types** using info ().

```
    print(df.dtypes)                          ##To display datatypes of each column

    transaction_id         object
    timestamp              object
    customer_id            object
    customer_name          object
    age                   float64
    city                   object
    state                  object
    country                object
    transaction_type       object
    category               object
    amount                float64
    currency               object
    account_balance       float64
    bank_name              object
    merchant_id            object
    device_type            object
    transaction_status     object
    credit_score            int64
    kyc_level              object
    risk_level             object
    is_fraud                 bool
    dtype: object
```

- Correct the data types

```
##Changed the datatypes
df['Date & time'] = pd.to_datetime(df['Date & time'],errors='coerce')

df['Age'] = df['Age'].astype(int)

df['Credit_score'] = df['Credit_score'].astype(int)

df['Is_fraud'] = df['Is_fraud'].astype(bool)
```

- Checked for **missing/null values** in the dataset.

```
print(df.isnull())                          ##To check for null values in the dataset
```

```
      transaction_id  timestamp  customer_id  customer_name    age   city  \
0              False      False        False          False  False  False
1              False      False        False          False  False  False
2              False      False        False          False  False  False
3              False      False        False          False  False  False
4              False      False        False          False  False  False
...              ...        ...          ...            ...    ...    ...
4995           False      False        False          False  False  False
4996           False      False        False          False  False  False
4997           False      False        False          False  False  False
4998           False      False        False          False  False  False
4999           False      False        False          False  False  False

      state  country  transaction_type  category  ...  currency  \
0     False    False             False     False  ...     False
1     False    False             False     False  ...     False
2     False    False             False     False  ...     False
3     False    False             False     False  ...     False
4     False    False             False     False  ...     False
...     ...      ...               ...       ...  ...       ...
4995  False    False             False     False  ...     False
4996  False    False             False     False  ...     False
4997  False    False             False     False  ...     False
4998  False    False             False     False  ...     False
4999  False    False             False     False  ...     False
```

  - Calculate the number of missing/null values in the dataset

```
print(df.isnull().sum())              ##To count the null values of each row
```

```
transaction_id        0
timestamp             0
customer_id           0
customer_name         0
age                   4
city                  0
state                 0
country               0
transaction_type      0
category              0
amount                4
currency              0
account_balance       2
bank_name             0
merchant_id           0
device_type           0
transaction_status    0
credit_score          0
kyc_level             0
risk_level            0
is_fraud              0
dtype: int64
```

- Depending on data, I replace missing values using median

```
num_columns = ['age', 'amount', 'account_balance' ]

for cols in num_columns:
  df[cols] = df[cols].fillna(df[cols].median())
```

- Checked for **duplicate records**.

```
print(df.duplicated())
```

```
0       False
1       False
2       False
3       False
4       False
        ...
4995    False
4996    False
4997    False
4998    False
4999    False
Length: 5000, dtype: bool
```

```
print(df.duplicated().sum())                ##To count the number of duplicates in the dataset
```

```
0
```

```
# The dataset has no duplicate values
# df = df.drop_duplicates()
```

- To check & Treat the Outliers in numeric columns

```
for col in num_cols:

    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    # Count outliers BEFORE capping
    outlier_count = ((df[col] < lower) | (df[col] > upper)).sum()

    print(f"{col:<20} outliers before capping: {outlier_count}")

    # Apply IQR capping
    df[col] = df[col].clip(lower=lower, upper=upper)

print("Outliers treated for all numeric columns!")
```

```
Age                  outliers before capping: 0
Amount               outliers before capping: 515
Account_balance      outliers before capping: 460
Credit_score         outliers before capping: 26
Outliers treated for all numeric columns!
```

Outlier Detection and Treatment

Outliers were identified in the numerical columns using the **Interquartile Range (IQR) method**. The analysis identified **515 outliers in the Amount**

column, 460 outliers in the Account Balance column, and 26 outliers in the Credit Score column, while no outliers were detected in the Age column.

```
df['Amount_log'] = np.log1p(df['Amount'])

df['Account_balance_log'] = np.log1p(df['Account_balance'])

##check transformation
print("Amount:", df['Amount'].skew())
print("Amount_log:", df['Amount_log'].skew())

print("Account_balance:", df['Account_balance'].skew())
print("Account_balance_log:", df['Account_balance_log'].skew())
```

```
Amount: 1.1914861721248773
Amount_log: -0.3854604727682383
Account_balance: 1.1617013395805076
Account_balance_log: -0.3034462604065687
```

- The Amount and Account Balance variables showed high variability and skewness due to extreme values. Therefore, appropriate **transformation techniques were applied to reduce skewness and minimize the influence of extreme values**. After transformation, IQR-based capping was applied to control the remaining extreme observations and prepare the data for further analysis

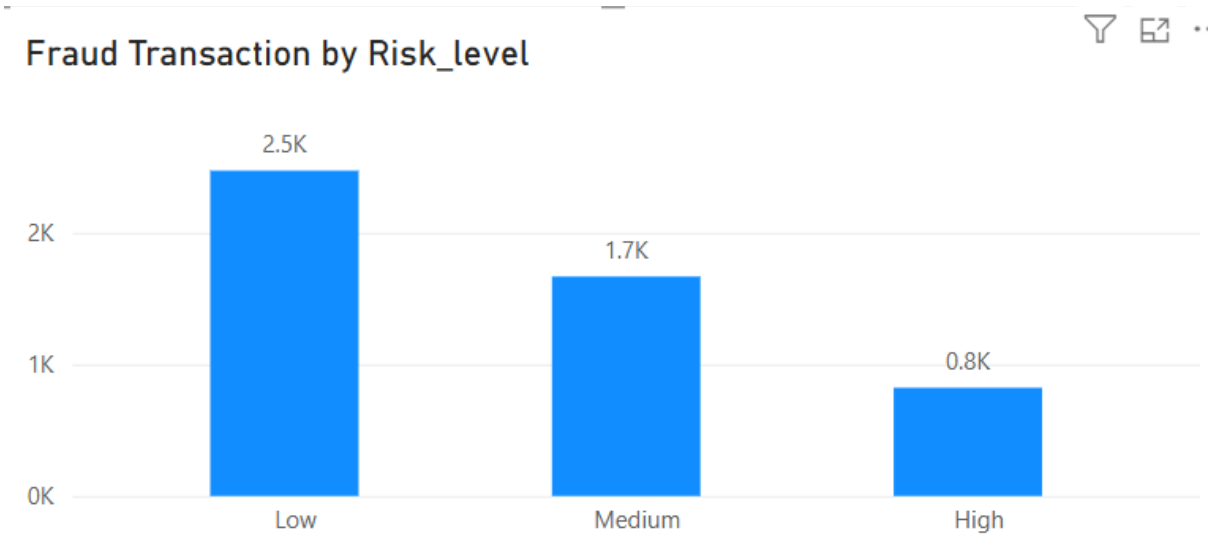
- Prepared the dataset for further **analysis and visualization**.

6. Tools & Technologies

- Microsoft Excel
- Power bi
- Power Query
- DAX
- Python
- NumPy
- Pandas
- Seaborn
- ChatGpt

7. Performance Analysis

1. Fraud Transaction by Risk Level

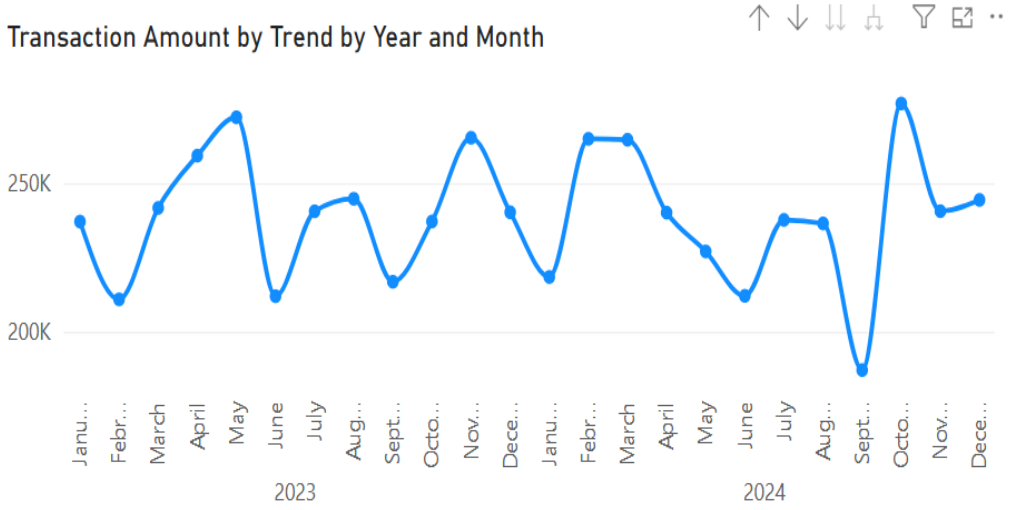


- Insight:
  The chart shows that fraudulent transactions are higher in the Low and Medium risk levels, while High risk has fewer fraudulent transactions.

- Business Recommendation:

The business should improve risk classification and closely monitor Low and Medium risk transactions to identify hidden fraud.

2. Transaction Amount by Trend by Year and Month



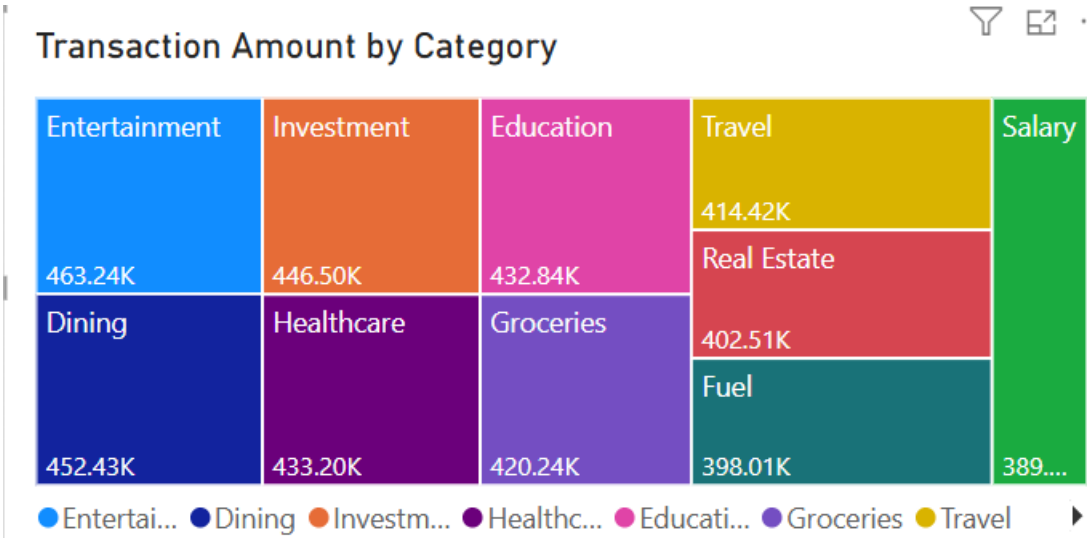
- Insight:

The transaction amount fluctuates over time, with several increases and decreases. There is no consistent upward trend.

- Business Recommendation:

The business should monitor monthly transaction patterns and investigate sudden increases or decreases to identify unusual activity.

3. Transaction Amount by Category



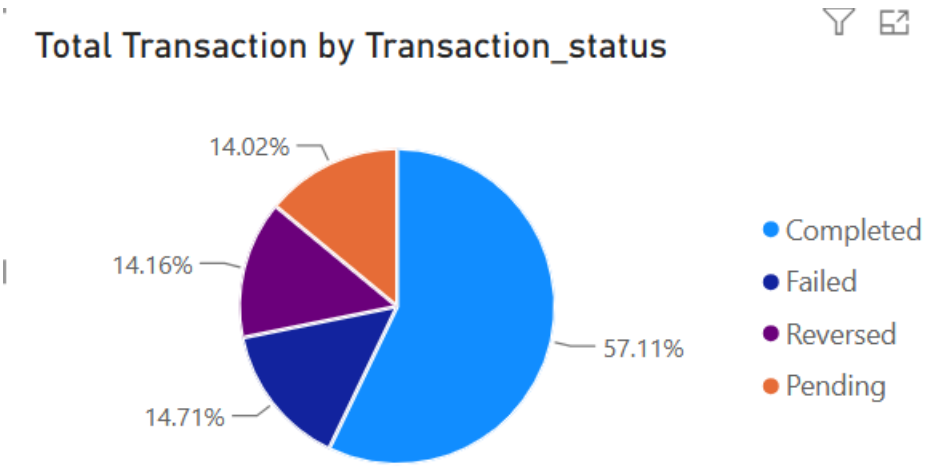
- Insight:

The transaction amount varies across different categories. Some categories contribute more transaction value than others.

- Business Recommendation:

The business should focus on high-value categories and monitor unusual transaction amounts to reduce financial risk.

4. Total Transaction by Transaction Status



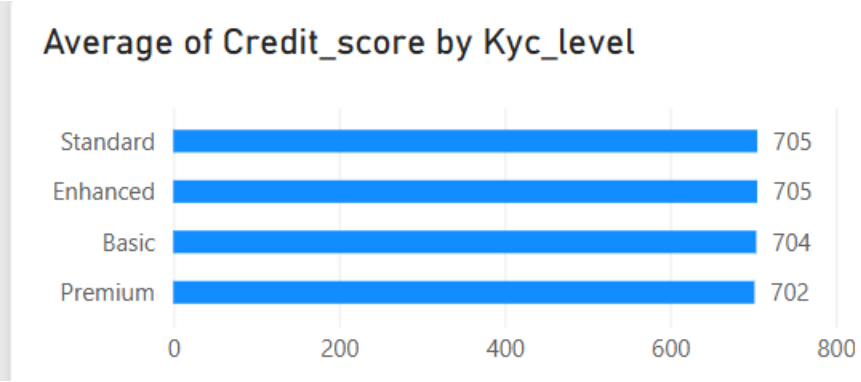
- Insight:

Most transactions are completed successfully, while failed, pending, and reversed transactions represent smaller portions.

- Business Recommendation:

The business should monitor failed and reversed transactions to identify technical issues and reduce unsuccessful transactions.

5. Average of Credit Score by KYC Level

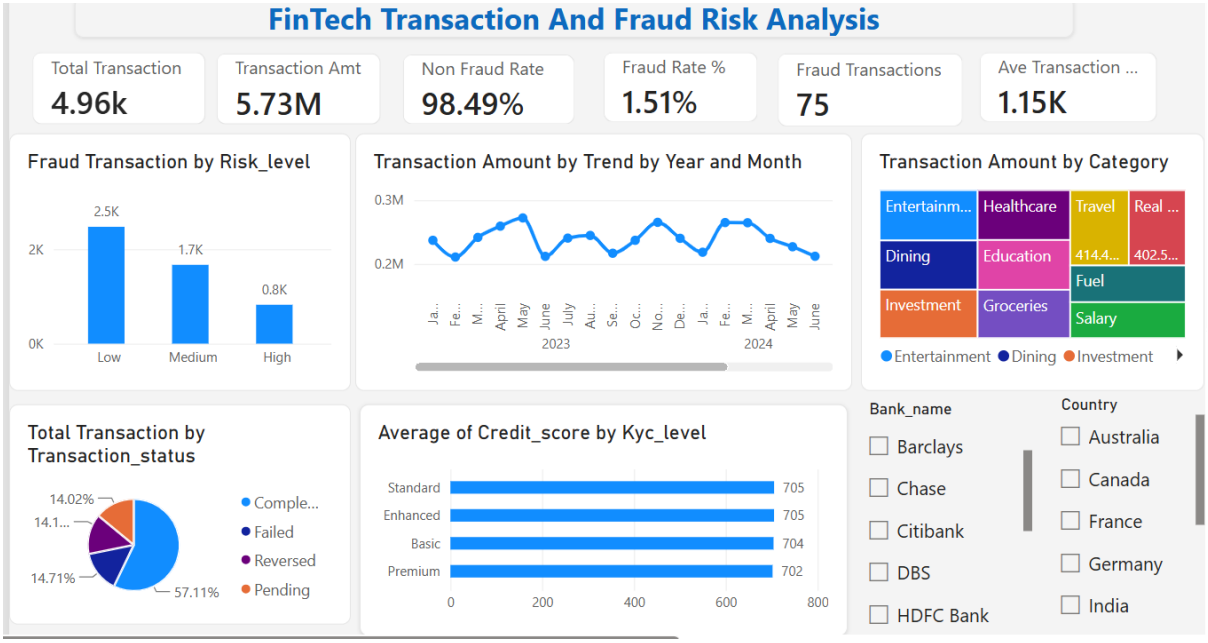


- Insight:

The average credit scores are almost similar across the different KYC levels, showing that there is no major difference in average credit score between the groups.

- Business Recommendation:

The business should not rely only on credit score. It should combine credit score with KYC level, transaction behavior, and risk level for better risk assessment.



Overall Dashboard Insights

- The dashboard shows that fraudulent transactions are present across different risk levels.
- Transaction amounts fluctuate over time, with some periods showing higher activity.
- Transaction values vary across different transaction categories.
- Most transactions are successful, while a smaller number are failed, pending, or reversed.
- Credit scores are relatively similar across different KYC levels.

Overall Business Recommendations

- The business should continuously monitor transactions and strengthen fraud detection methods.
- Unusual changes in transaction amounts should be investigated to identify potential risks.
- High-value transaction categories should be monitored more closely.

- The business should investigate failed and reversed transactions to improve transaction success.
- Credit score should be combined with KYC, transaction behavior, and risk level for better fraud-risk assessment.